

Verification Engineer: Saksham Bansal

Discord: bansalsaksham

GT: sbansal91

Functional Tests:

Test cases that cover all major functionality of the module. Each test must clearly specify what behavior is being tested, the input stimulus sequence, and expected outputs/pass criteria

Test Case	Description	Input Sequence	Expected Output
1.1	Memory Initialization	Pulse initialize signal for all 1024 SRAM addresses.	Scoreboard updates internal model with matching values.
1.2	Standard 64-bit Addition	Load SRAM A/B with random 32-bit values; trigger start_calc.	LSB and MSB summed with carry propagation to destination.
1.3	Monitor/SB Sync	Execute sequential calculations from different address ranges.	Monitor samples SRAM read latency correctly for Scoreboard check.

Corner Cases:

Test cases that verify boundary conditions, illegal inputs, timing edges, or rare conditions.

Test Case	Description	Input Sequence	Expected Output
2.1	Zero Addition	Initialize SRAM values to 32'h0000_0000.	Destination SRAM receives 32'h0 for both halves.
2.2	Maximum Overflow	Initialize SRAM values to 32'hFFFF_FFFF.	Correct 64-bit sum with MSB carry-out verified.
2.3	Mid-Flight Reset	Toggle rst_i while FSM is in S_CALC_LO state.	FSM instantly returns to S_IDLE via asynchronous reset.

Constrained Randomized Testing:

*Random or semi-random tests with defined constraints to improve coverage. Note: Coverage Requirement is **98%***

Test Case	Description	Input Sequence/Constraint	Expected Output
3.1	Random Address Blocks	2000 iterations of random read_start and write_start pairs.	Validates RTL stability across the entire memory map.
3.2	Toggle Coverage Sweep	Force out-of-bounds addresses: 0xAAAA_AAAA, 0x5555_5555, 0xFFFF_FFFF.	Achieves 99.58%+ toggle coverage on 32-bit buses.
3.3			

Assertions:

List key assertions used to check protocol rules, timing rules, state transitions, etc. in the design.

Test Case	Description	Assertion Property
4.1	Async Reset Protocol	property (@(posedge clk) (rst -> (state == S_IDLE)))
4.2	LSB to MSB Sequence	property (@(posedge clk) (state == S_CALC_LO) => (state == S_CALC_HI))
4.3	Arithmetic Carry Integrity	property (@(posedge clk) (state == S_CALC_LO && carry_out) => (state == S_CALC_HI && carry_in))